Savitribai Phule Pune University
Centre For Distance and Online Education

SAVITRIBAI PHULE PUNE UNIVERSITY, PUNE

CENTRE FOR DISTANCE AND ONLINE EDUCATION

Program	**MCA**	
Course	**Programming from First Principles**	
Topic Name	**Use of first-class functions Practical Session**	
Topic Code	**-**	
Unit/Module No. & Name	**5**	**Higher Order Functions**
Self-Learning Hours	**-**	

Topic Details

© SPPU | Original Content | All Rights Reserved Copying or Reposting Without Permission is Strictly Prohibited

OBJECTIVE

1. This self-learning material explains the practical use of higher order functions in Python.
2. It begins with a simple revision of iterables and iterators and then moves to the practical working of built-in higher order functions such as map, filter, reduce, any, and all.
3. The lesson uses simple examples to show how these functions work with user-defined functions, lambda functions, and different iterables such as list, set, and tuple.
4. By the end of this lesson, students should be able to write and test practical Python programs using higher order functions with confidence.

1.0 INTRODUCTION

In Python programming, higher order functions are very useful because they help process data in a short, elegant, and flexible way. Before using them in practical programs, students must understand how iterables and iterators work. This is important because higher order functions usually take an iterable as input and then apply some logic to its elements.

The practical session begins with a simple list of characters to explain how an iterator is created and how one value is accessed at a time. After that, the lesson moves to map, filter, reduce, any, and all. These functions are not only theoretical ideas. They are practical Python tools that help transform, select, combine, and test values inside collections.

The lesson also shows that higher order functions can work with user-defined functions as well as lambda functions. It demonstrates their use on different iterables such as lists, sets, and tuples. This makes the practical understanding stronger and helps students learn how to apply the same idea in multiple forms.

2.0 MEANING AND DEFINITION

2.1 Meaning Of Practical Higher Order Functions

Practical higher order functions are Python functions that work on other functions and iterables during actual program execution. Instead of only studying their definition, practical learning shows how they behave with real inputs and outputs. This helps students understand how a function transforms data, filters values, reduces a collection to one answer, or checks truth values in an iterable.

In simple words, practical higher order functions show how abstract programming ideas are used in real code. They help students move from theory to application and understand the working logic through examples.

2.1.1 Formal Definition

In academic terms, practical higher order functions are functional programming constructs in Python that accept a function and one or more iterables as input and return either an iterable object, a filtered collection, a reduced single value, or a truth-based result. Their practical use helps build concise, modular, and reusable code.

2.2 Main Ideas In The Definition

The definition includes some key ideas. First, these functions work on iterables. Second, they often take another function as input. Third, their output may be an iterable or a single value. Fourth, they improve code quality by reducing repetition. Fifth, practical examples help students understand their actual use in Python.

3.0 ITERABLES AND ITERATORS IN PRACTICE

3.1 Practical Meaning Of Iterables

An iterable is a collection of values that can be accessed one element at a time in Python. In practical programming, iterables are very common because they allow many values to be stored together under one name. Examples of iterables include list, tuple, set, and dictionary. In the practical session, a list containing characters such as a, b, and c is used to explain the idea clearly. These values are stored in order and can be taken one after another when needed. This same concept can also be applied to tuples, sets, and dictionaries, though their internal behavior may be slightly different. Iterables are important because higher order functions such as map and filter need data in iterable form. Without iterables, these functions would not have a collection of elements to process. Therefore, iterables form the basic structure on which many Python operations are performed.

3.1.1 Examples Of Iterables In Python

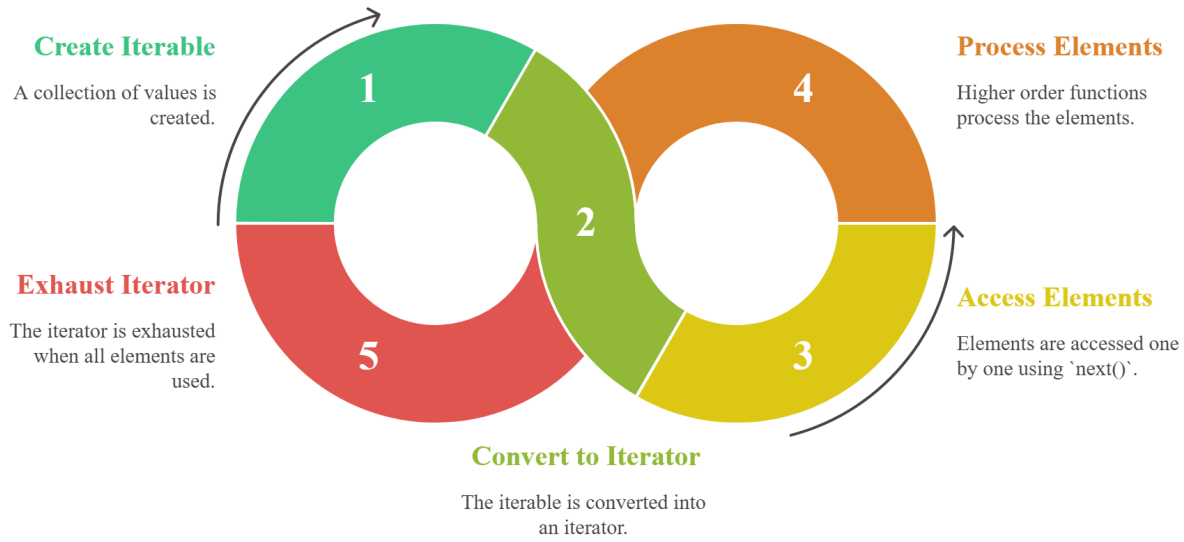
Python provides many built-in iterable data structures, and each of them is useful in different situations. A list is an ordered collection that can store multiple values and allows changes after creation. A tuple is also ordered, but it is fixed once it is created. A set stores unique values and does not maintain repeated elements. A dictionary stores data in key-value pairs and can also be traversed step by step. In the practical explanation, a list is first used because it is simple and easy to understand. However, the same iteration idea works for tuples, sets, and dictionaries as well. This shows that iterables are not limited to only one type of container. In Python programming, any collection that can be moved through one element at a time is treated as an iterable, and this makes them very important in both simple and advanced code.

3.2 Practical Meaning Of Iterators

An iterator is an object created from an iterable that allows the program to access elements one by one in sequence. It is generated by using the iter() function on an iterable. Once the iterator is created, Python does not give all values at once. Instead, it gives one value at a time using the next() function. In the practical example, the iterator created from the list first returns a, then b,

and then c when next() is called repeatedly. This process continues until all values are used. At that point, the iterator is exhausted. The iterator concept is important because it shows the actual mechanism by which Python moves through a collection. Rather than thinking of a list only as stored data, students begin to understand how Python accesses and processes each member step by step. This makes iterators a central idea in Python traversal and functional programming.

Fig. 1 Iterables and Iterators Cycle



This visual explains the iterable and iterator cycle, showing steps from creating an iterable, converting it into an iterator, accessing elements sequentially, processing them, and finally exhausting the iterator.

3.2.1 Working Of iter() And next()

The functions iter() and next() are the two main tools used in practical iterator handling. The iter() function converts an iterable into an iterator. This means it prepares the collection for step-by-step access. After that, the next() function is used to get the next available value from the iterator. Each time next() is called, the iterator moves forward and returns the following element. For example, if the iterable contains a, b, and c, the first call to next() returns a, the second returns b, and the third returns c. If next() is called again after the last value, the iterator has no more elements to return. This working process is very useful for understanding how loops and

higher order functions behave internally. It also helps students see that iteration is not random. It is an ordered process controlled by the iterator object in Python.

3.3 Importance In Practical Programming

The iterator concept is important in practical programming because it shows how Python handles collections step by step. This understanding is very helpful for students when they begin to work with loops, traversals, and higher order functions. Instead of thinking that Python reads all values at once, students learn that values are often accessed one after another in a controlled order. This makes the working of map, filter, and similar functions much easier to understand. These functions depend on iterable input, and behind the scenes they move through the elements in the same stepwise way. Iterators therefore help in building a deeper understanding of Python processing. They also make programming more efficient because values can be handled one at a time. Thus, the concept of iterables and iterators is not only theoretical. It has strong practical value in writing clear, correct, and efficient Python programs.

3.3.1 Role In Higher Order Functions

Iterables and iterators play an important role in higher order functions because these functions depend on collections of values for their operation. When we use map, filter, or reduce, Python takes the iterable and processes its elements one by one. This stepwise access is possible because the iterable can be turned into an iterator internally. For example, map applies a function to each element in sequence, filter checks each element one by one, and reduce combines values progressively. If students understand iterators, they can better understand how these higher order functions actually work. This makes their programming knowledge stronger and more practical. It also helps them write code with more confidence because they know what is happening inside the function calls. Therefore, iterables and iterators are not only useful on their own, but they are also essential for understanding functional programming tools in Python.

4.0 MAP FUNCTION IN PRACTICE

4.1 Practical Meaning Of Map

The map function is one of the most useful built-in higher order functions in Python. It takes two main arguments: a function and an iterable. After receiving these, it applies the function to each element of the iterable one by one. The result is returned as a map object, which can later be converted into a list or another iterable form for display. In practical programming, map is helpful when the same operation must be performed on every value in a collection. This removes the need to write repeated loops for similar work. In the practical session, the cube function is first used as a user-defined function. After that, the same operation is performed by using a lambda function. This comparison helps students understand both methods clearly. It also shows that the map function is flexible, compact, and very useful in Python when many values need the same type of transformation.

4.1.1 Importance Of Map In Practical Programming

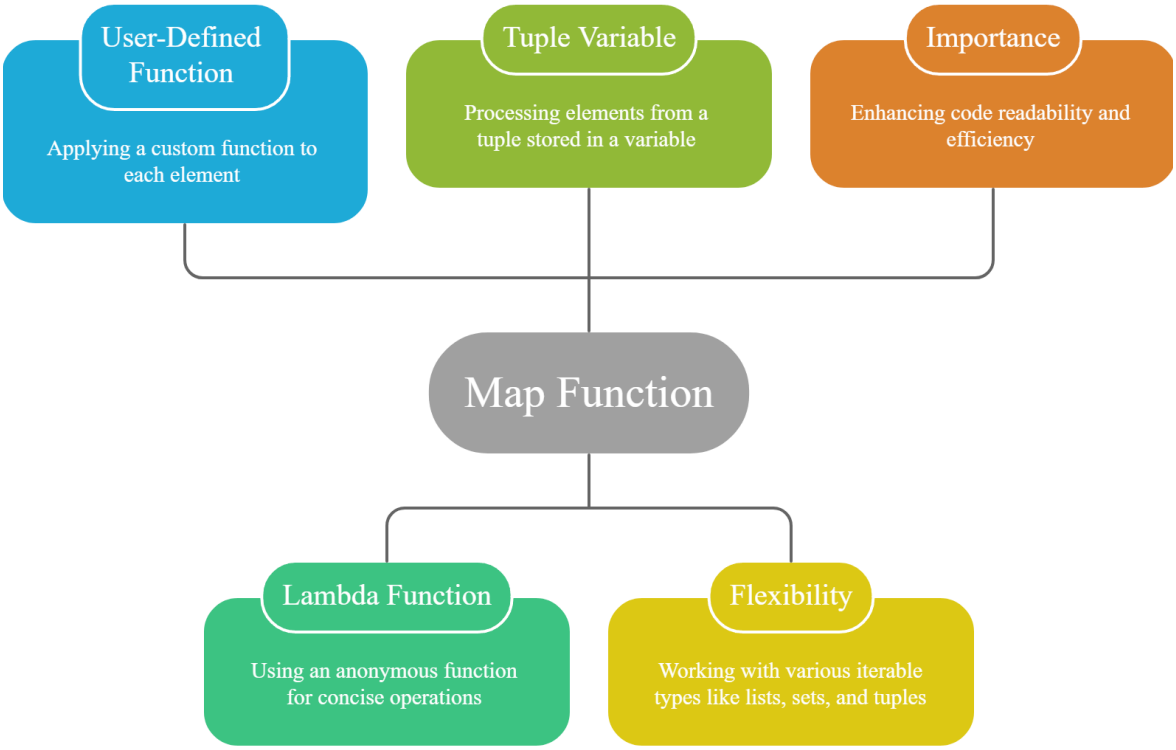
The map function is important in practical programming because it helps programmers apply one operation to all elements of a collection in a very clean and efficient way. Instead of writing a loop and repeatedly updating values, the programmer can directly use map with the required function and iterable. This improves readability because the purpose of the code becomes clear immediately. It also reduces code length and avoids repetition. Another important point is that map supports both user-defined functions and lambda functions, which makes it flexible for different programming situations. It can also work with different iterables such as lists, sets, and tuples. Because of this, the map function becomes a practical tool for transforming data quickly. For students, it is especially useful because it shows how Python combines functions and collections to produce new results in a simple and elegant form.

4.2 Map With User-Defined Function

In the practical example, a user-defined function cube(x) is created first. This function takes one argument and returns the cube of that value by calculating x to the power 3. After checking that the function works properly on a single value, it is passed to the map function along with a list

containing numbers such as 2, 7, and 3. The map function then applies cube to each element of the list one after another. Since map does not directly return a list, the result first appears as a map object. Therefore, it is converted into a list before printing. Once converted, the output becomes the cubes of the given numbers, such as 8, 343, and 27. This practical example is useful because it shows how a normal named function can be reused with map to process each element of an iterable. It also makes the working of map very easy to understand.

Fig. 2 Practical Applications of the Map Function



This diagram illustrates practical applications of the Python map function, highlighting its use with user-defined and lambda functions, tuple processing, flexibility across iterables, and its role in improving code readability and efficiency.

4.2.1 Understanding The Output Of Map

One important point in practical use of map is understanding its output. When the map function is executed, it does not immediately display all transformed values as a list. Instead, it returns a map object. This object stores the transformation process and can later be converted into a list,

tuple, or another iterable form depending on the need. In the practical example, after applying the cube function to the list [2, 7, 3], the result is converted into a list so that the transformed values can be printed clearly. The output becomes [8, 343, 27]. This shows that map acts on every element in sequence and creates a new iterable containing the results. For students, this is important because it explains why map alone shows an object reference, while converting it into a list makes the actual result visible. Thus, understanding the output form is an essential part of using map correctly.

4.3 Map With Lambda Function

The same practical task can also be performed by using a lambda function instead of a user-defined function. In this case, the cube logic is written directly inside the map function as a lambda expression. The lambda takes one argument and returns its cube in a single expression. A set containing values such as 3, 4, 5, and 6 is then passed as the iterable. The map function applies the lambda to each element of the set and again returns a map object. This output is converted into a list before printing. The result contains the cubes of the values in the set. This practical example is useful because it shows that a separate named function is not always necessary. When the logic is small and temporary, lambda makes the code shorter and more direct. It also shows that map is flexible enough to work with sets along with other iterable types.

4.3.1 Benefits Of Using Lambda With Map

Using a lambda function with map has several practical benefits. First, it saves time because the programmer does not need to write a separate function with def for small operations. Second, it makes the code shorter and more direct when the required logic is simple. Third, it is very useful when the function is needed only once and does not have to be reused elsewhere in the program. In the practical session, the cube calculation is written directly as a lambda expression inside map. This shows that Python supports compact programming for simple tasks. Lambda and map together are a common combination in functional programming style. They help the programmer express the transformation clearly without extra lines of code. For students, this example is useful because it shows an alternative method to achieve the same result. Thus, lambda with map is a practical and efficient coding technique in Python.

4.4 Map With Tuple Variable

In the next practical example, a tuple is first stored in a separate variable and then passed to the map function. The tuple contains values such as 12, 15, and 20. The same user-defined cube function is used again. When map is called with the cube function and the tuple variable, it applies the cube operation to each tuple element in sequence. Just like in the earlier cases, the output is a map object first. It is then converted into a list for display. The resulting list contains the cubes of 12, 15, and 20. This practical example shows that map is not limited to directly written lists or sets. It can also work with iterables that are stored in variables before being passed. This makes the use of map more realistic in programming, because data is often stored in variables before processing. Therefore, this example strengthens the understanding that map can work with many iterable forms.

4.4.1 Flexibility Of Map With Different Iterables

One important lesson from the practical examples is that the map function is flexible with respect to the type of iterable it can process. It can work with lists, sets, tuples, and other iterable objects as long as their elements can be accessed one by one. In the practical session, map is applied first to a list, then to a set with a lambda function, and finally to a tuple stored in a variable. This shows that the logic of map remains the same even when the iterable changes. Only the source of values is different. This flexibility makes map a powerful and reusable function in Python programming. It allows the same operation to be performed on different collections without changing the main logic. For students, this is an important practical idea because it shows that once the concept of map is understood, it can be applied widely across different data structures in Python.

5.0 FILTER FUNCTION IN PRACTICE

5.1 Practical Meaning Of Filter

The filter function is used when we want to keep only those values that satisfy a condition. It takes a function and an iterable as input. The function must return either true or false. Only those elements for which the result is true are included in the output. This is why filter acts like a selector.

5.2 Filter With User-Defined Function

In the first practical example, a function is_positive(x) is written to check whether a number is greater than zero. The function is first tested with positive and negative values. Then it is passed to filter along with a tuple containing both positive and negative numbers. The output is converted into a list and printed. Only the positive values remain in the result.

5.3 Filter With Lambda Function

The next filter example uses a lambda function instead of a separate named function. The condition checks whether a number is a multiple of 5 by using n mod 5 equals 0. A list with mixed values is passed to filter, and the output contains only those values that are multiples of 5. This includes both positive and negative values if they satisfy the condition.

5.4 Practical Understanding Of Filter

These practical examples show that filter does not change the values. It only selects the values that satisfy the condition. This makes it very useful when students need to extract specific items from a larger dataset.

6.0 REDUCE FUNCTION IN PRACTICE

6.1 Practical Meaning Of Reduce

The reduce function is used to combine all elements of an iterable into a single value. It works differently from map and filter because it does not return another iterable. Instead, it processes the elements step by step until only one final answer remains. Since reduce is not directly available like map and filter, it must be imported from functools.

6.2 Reduce With User-Defined Function

In the practical example, a function multiply(m, n) is defined to return the multiplication of two values. This function is passed to reduce along with a set of numbers. Before importing reduce, Python shows an error saying that reduce is not defined. After importing it from functools, the function works correctly and combines the values into one final product.

6.3 Working Process Of Reduce

The practical explanation shows that reduce first combines the first two elements. Then it combines that result with the next element. This process continues until the iterable is fully processed. Thus, the final output becomes one single value.

6.4 Reduce With Lambda Function

The session also explains that reduce can be used with a lambda function when the logic is small and used only once. This saves time and avoids writing a separate named function.

7.0 ANY FUNCTION IN PRACTICE

7.1 Practical Meaning Of Any

The any function checks whether at least one value in the iterable is true. If one or more values are true, the result is true. If all values are false, or if the iterable is empty, the result is false. This makes any useful for checking whether at least one required condition is satisfied.

7.2 Prime Example With Any

In the practical session, a prime-checking function is first created. It tests whether a number is prime and returns true or false. This function is first checked on individual values and then applied to a list using map. The result is a list of truth values. This list is then passed to any. If at least one value is true, the output of any is true.

7.3 All False And Empty List Cases

The lesson also shows two important cases. If all mapped values are false, then any returns false. If filter generates an empty list because no value satisfies the condition, then any also returns false. These cases help students understand the exact working of the function.

8.0 ALL FUNCTION IN PRACTICE

8.1 Practical Meaning Of All

The all function checks whether every value in an iterable is true. It returns true only if all values satisfy the condition. If even one value is false, the result becomes false. This makes all stricter than any because it requires complete truth across the iterable.

8.2 Prime Example With All

The practical session uses the same prime-checking function for all. A set of prime numbers is first passed through map, and the resulting truth values are then given to all. Since all results are true, the function returns true. Later, one non-prime value is added to the set, and the output changes to false.

8.3 Practical Understanding Of All

This example clearly shows that all is useful when every single value must satisfy the condition. It is often used in validation and strict logical checking.

9.0 ADVANTAGES OF PRACTICAL HIGHER ORDER FUNCTIONS

9.1 Major Advantages

The practical use of higher order functions shows many advantages. They reduce code length, improve readability, and make logic more modular. They also help programmers avoid repeated loops and conditions. Since they work with user-defined functions as well as lambda functions, they provide flexibility in code design.

9.2 Practical Importance For Learners

For students, practical examples are especially important because they show the direct relation between function definition, iterable input, and actual output. This improves understanding and confidence in Python programming.

1. Iterators are created from iterables using iter().
2. next() accesses one value at a time from an iterator.
3. map applies a function to every element of an iterable.
4. filter selects only the elements that satisfy a condition.
5. reduce combines values into a single output.
6. reduce must be imported from functools.
7. any returns true if at least one value is true.
8. all returns true only when every value is true.
9. User-defined functions and lambda functions can both be used with higher order functions.
10. Lists, sets, and tuples can all be used as iterables in practical Python code.

10.0 KEYWORD TABLE

Keyword	Touch Vocabulary
Iterable	A collection that can be traversed
Iterator	An object used to access values one by one
iter()	Function used to create an iterator
next()	Function used to get the next value
Higher Order Function	A function working on another function
map	Function for transforming all elements
filter	Function for selecting matching elements
reduce	Function for combining all elements
functools	Module used for reduce

lambda	Anonymous short function
Prime Function	Function that checks whether a number is prime
User-Defined Function	A function created by the programmer
Truth Value	True or false result
Modular Code	Code divided into reusable parts
Functional Programming	Programming style using functions as values